IDC 222 - Practice Questions (STL & Templates)

---

1. Function Templates for Math Operations

Write a C++ program with the following function templates:

- `power()` - calculates base raised to exponent (exponent is integer)
- `absolute()` - returns absolute value
- `percentage()` - calculates percentage of a value

Test with integer and float types.

Example:

Input:

```
Enter base and exponent (Integer): 5 3
Enter a number (Integer): -45
Enter value and percentage (Integer): 200 15

Enter base and exponent (Float): 2.5 3
Enter a number (Float): -34.56
Enter value and percentage (Float): 150.5 20
```

Output:

```
=== Integer Operations ===
5 raised to power 3 = 125
Absolute value of -45 = 45
15% of 200 = 30

=== Float Operations ===
2.5 raised to power 3 = 15.625
Absolute value of -34.56 = 34.56
20% of 150.5 = 30.1
```

---

2. Function Templates for Unit Conversions

Write a C++ program using function templates to perform the following operations:

- `convertLength()` - converts meters to kilometers
- `convertTemperature()` - converts Celsius to Fahrenheit
- `convertWeight()` - converts kilograms to grams

Test with integer and float types.

Example:

Input:

```
Enter length in meters (Integer): 1500
Enter temperature in Celsius (Integer): 25
Enter weight in kilograms (Integer): 5

Enter length in meters (Float): 1234.5
Enter temperature in Celsius (Float): 36.6
Enter weight in kilograms (Float): 2.75
```

Output:

```
=== Integer Conversions ===
1500 meters = 1.5 kilometers
25 Celsius = 77 Fahrenheit
5 kilograms = 5000 grams

=== Float Conversions ===
1234.5 meters = 1.2345 kilometers
36.6 Celsius = 97.88 Fahrenheit
2.75 kilograms = 2750 grams
```

---

3. Rectangle Template Class

Write a C++ program with a class template `Rectangle<T>` where T is type for dimensions:

Member variables:

- `length` (T type)
- `width` (T type)

Member functions:

- `setDimensions()` - sets length and width
- `getLength()` and `getWidth()` - getter functions
- `calculateArea()` - returns area
- `calculatePerimeter()` - returns perimeter
- `isSquare()` - checks if it's a square
- `display()` - displays dimensions, area, and perimeter

Example:

Input:

```
Enter length and width (Integer): 10 5

Enter length and width (Float): 12.5 12.5

Enter dimensions for Rectangle 1: 10 8
Enter dimensions for Rectangle 2: 12 6
```

Output:

```
=== Rectangle with Integer Dimensions ===
Rectangle Details:
Length: 10
Width: 5
Area: 50
Perimeter: 30
Is Square? No

=== Rectangle with Float Dimensions ===
Rectangle Details:
Length: 12.5
Width: 12.5
Area: 156.25
Perimeter: 50
Is Square? Yes
```

---

4. Counter Template Class

Write a C++ program with a class template `Counter<T>` that maintains a counter:

Member variables:

- `count` (T type - can be int, float, or double)

Member functions:

- Constructor (initialize to zero)
- `increment()` - increases count by 1
- `decrement()` - decreases count by 1
- `incrementBy(value)` - increases count by given value
- `decrementBy(value)` - decreases count by given value
- `reset()` - sets count to zero
- `getCount()` - returns current count
- `display()` - displays current count

Test with integer counter and float counter.

Example:

Input:

```
Enter initial value (Float): 10.5
```

Output:

```
=== Integer Counter ===
Initial count: 0
After increment: 1
After increment: 2
After incrementing by 5: 7
After decrement: 6
After decrementing by 3: 3
Current count: 3
After reset: 0

=== Float Counter ===
Initial count: 10.5
After increment: 11.5
After incrementing by 3.2: 14.7
After decrement: 13.7
After decrementing by 2.5: 11.2
```

```
Current count: 11.2
After reset: 0
```

---

5. Student Marks using Vector

Write a C++ program using **vector** and **iterators only** to do the following:

- Read n student marks using `push_back()`.
- Display all marks using **iterators.**
- Insert a new mark at a user-given position using `insert()`.
- Remove the last mark using `pop_back()`.
- Remove a mark from a user-given position using `erase()`.
- Display the final marks using iterators only.
  Note: Do not use array indexing or range-based for loop.

Example:

Input:

```
Enter number of students: 5
Enter marks:
70 85 90 60 75
Enter a mark to insert: 88
Enter position to insert: 2
Enter position to delete: 3
```

Output:

```
Marks entered:
70 85 90 60 75
After inserting:
70 85 88 90 60 75
After deleting last mark:
70 85 88 90 60
Final marks:
70 85 88 60
```

---

6. Character Frequency Counter (Using Indexing)

Write a C++ program using **vector** with **indexing (`[]` or `at()`)** to:

- Read a sentence (string) and store each character in a vector.
- Display all characters using indexing.
- Read a character and count its frequency (case-sensitive).
- Count and display the number of alphabetic characters and digit characters.
  Note:
  - Use `getline()` to read the full sentence. If needed, use `ws` with getline() to handle whitespace.
  - Do not use `isalpha()` and `isdigit()` functions.

Example:

Input:

```
Enter a sentence: Happy New Year 2026
Enter character to count: a
```

Output:

```
Characters in vector: H a p p y   N e w   Y e a r   2 0 2 6
Frequency of 'a': 2
Alphabetic characters: 14
Digit characters: 4
```

---

7. Sensor Reading Search with Vector

Write a C++ program using **vector** container and **iterators** to:

- Read n sensor readings (floating-point values) into a vector.
- Display all readings with their **positions (0-indexed)** using iterators.
- Ask user for a target reading value and use iterators to **find all positions** where this value occurs.
- If found, display all positions; otherwise, display "Not found".
- Insert a new reading at a user-given position using `insert()`.
- Display the updated vector using **iterators.**
  Note: Do not use array indexing ([]) or range-based for loop.

Example:

Input:

```
Enter the number of readings: 6
Enter sensor readings:
```

```
23.5  19.8  23.5  30.2  23.5  18.4

Enter value to search: 23.5

Enter position to insert new reading: 3
Enter new reading value: 25.6
```

Output:

```
Position 0: 23.5
Position 1: 19.8
Position 2: 23.5
Position 3: 30.2
Position 4: 23.5
Position 5: 18.4

Value 23.5 found at positions: 0  2  4

Updated readings: 23.5  19.8  23.5  25.6  30.2  23.5  18.4
```

---

8. Student Grade Management using Vector (Indexing)

Write a C++ program using **two vectors** with **indexing (`[]` or `at()`)** to:

- Read n student grades (A, B, C, D, F) into the **first vector.**
- Display all grades using indexing.
- Count and display the frequency of each grade.
- Ask the user for a grade to remove and delete all its occurrences from the **first vector.**
- Convert the remaining grades into **Pass/Fail** and store the result in a **second vector**
    - A, B, C ⟶ **P**
    - D, F ⟶ **F**
- Display the pass/fail vector and count total passes and fails.
  Note: Do not use iterators.

Example:

Input:

```
Enter the number of students: 10
Enter grades:
A  B  A  C  D  A  B  F  C  A
Enter grade to remove: D
```

Output:

```
Grades entered: A  B  A  C  D  A  B  F  C  A

Grade frequency:
A: 4
B: 2
C: 2
D: 1
F: 1

Updated grades:   A  B  A  C  A  B  F  C  A

Pass/Fail status: P  P  P  P  P  P  F  P  P
Total Pass: 8
Total Fail: 1
```

---

9. Temperature Readings with List

Write a C++ program using `list` container to store daily temperature readings:

- Read n temperature values (in Celsius) into a list.
- Display all temperatures.
- Count and display how many temperatures are below freezing ($< 0^\circ C$).
- Remove a temperature from the front and one from the back.
- Ask for a range [min, max] and remove all temperatures in that range using iterators.
- Display the count of remaining temperatures and the updated list.

Example:

Input:

```
Number of days: 7
Temperatures: -5 12 8 -2 15 20 3
Range to remove [min max]: 5 15
```

Output:

```
Temperatures recorded: -5  12  8  -2  15  20  3
Temperatures below freezing: 2
After front/back removal: 12 8 -2 15 20
After range removal: -5 -2 20 3
Total temperatures remaining: 4
```

---

10. Menu-Driven Task Management using List (STL)

Write a menu-driven C++ program using the **list container** to manage a list of task names (strings).
The program should support the following operations:

- Add a task at the end of the list (`push_back()`)
- Add a task at the beginning of the list (`push_front()`)
- Insert a task at a given position (`insert()` with `advance()`)
- Delete the last task (`pop_back()`)
- Delete the first task (`pop_front()`)
- Remove a task by position (`erase()` with `advance()`)
- Remove a specific task by name (`remove()`)
- Display all tasks using **iterators**

Example: Input/Output

```
----- MENU -----
1. Add task at end
2. Add task at beginning
3. Insert task at position
4. Delete task from end
5. Delete task from beginning
6. Delete task by position
7. Remove task by name
8. Display all tasks
0. Exit
---------------
```

```
Enter your choice: 1
Enter task name: Homework
Enter your choice: 1
Enter task name: Project
Enter your choice: 2
Enter task name: Meeting
Enter your choice: 8
Tasks: Meeting Homework Project
Enter your choice: 6
Enter position to delete: 2
Task deleted using erase().
Enter your choice: 8
Tasks: Meeting Project
Enter your choice: 7
Enter task name to remove: Homework
Task not found.
Enter your choice: 8
Tasks: Meeting Project
Enter your choice: 0
Exiting program.
```

---

11. Name Management with List

Write a C++ program using **list** container and **iterators** to manage a list of names:

- Read **n** names into a `list` using `push_back()`.
- Display all names using a **range-based for loop.**
- Ask for a name to search and display its **position** (1-indexed) if found, otherwise print "Not found".
- Ask for a character and display all names starting with that character.
- Demonstrate deletion by removing the **first and last name** using **pop_front()** and **pop_back().**
- Display the final list using a **range-based loop.**

Example:

Input:

```
Number of names: 6
Names: Alice Bob Charlie Anna David Amy
Name to search: Charlie
Character to filter by: A
```

Output:

```
Names: Alice Bob Charlie Anna David Amy
Name 'Charlie' found at position: 3
Names starting with 'A': Alice Anna Amy
Final list after removing first and last: Bob Charlie Anna David
```
